



JAM

JOB APPLICATION MANAGER - BY FAROUK

Complete Technical Documentation

Development snapshot 0.1.0

Architecture, history, features, data model, Analyzer, tests, Windows release plan, GitHub strategy and continuity handoff

State consolidated on 23 September 2026 - Development version 0.1.0

JAM is a local-first Windows desktop application for managing job applications. This document captures the current implementation through Batch 3, the Analyzer v4 state, known defects, release strategy, and the exact context required to continue safely in a new conversation.

Application:	JAM - Job Application Manager
Development version:	0.1.0
Author / developer:	Ahmed-Farouk DAMERGI (Farouk)
Website:	https://www.damergi.com
Native project format:	.jam
Target platform:	Windows 10/11 x64
Development folder:	C:/Users/fdame/Desktop/JAM
Source repository:	Planned later; expected separate source repository
Public repository:	Planned later; user-facing distribution only

Contents

1	Executive summary and current state	1
1.1	Product boundary	1
1.2	Current priorities	1
2	Product identity and design system	1
2.1	Brand family	1
2.2	Core palette	2
2.3	Branding assets	2
2.4	UX rules	2
3	Technical architecture	2
3.1	Technology stack	3
3.2	Runtime architecture	3
3.3	Separation of responsibilities	3
4	Runtime data, storage and project portability	3
4.1	Default runtime locations	4
4.2	SQLite schema	4
4.3	Migration philosophy	4
4.4	Portable .jam projects	4
5	Current application modules	5
6	Applications and pipeline behavior	5
6.1	Application fields	5
6.2	Structured salary	6
6.3	Settings salary converter	6
7	Batch 1 - foundation and safety	6
7.1	Notable Batch 1 regressions that were fixed	7
8	Batch 2 - application management and workflow	7
8.1	Excel import behavior	7
8.2	Critical Windows import crash and resolution	7
9	Batch 3 - richer tracking and Capture improvements	8
9.1	Multiple contacts	8
9.2	Notes history	8
9.3	Attachments	8
9.4	Tracking UI placement	8
9.5	Capture Mode persistence	8
10	Capture Mode architecture	9
11	Analyzer architecture and current defect	9
11.1	Analyzer v3 foundation	9
11.2	Analyzer v4 hybrid layer	10
11.3	Real-world Klépierre test on 23 September 2026	10
11.4	Root cause	11
11.5	Analyzer v4.1 - next required fix	11
11.6	Dataset strategy	11
12	Analyzer report output	12
13	Import, export and project workflow	12
13.1	Excel export	12
13.2	Excel import	12
13.3	PDF export	12
13.4	Export history	13
14	Settings, internationalization and diagnostics	13
14.1	Language	13
14.2	Data Safety	13
14.3	Data Locations	13
14.4	Diagnostics	13
15	Testing and regression discipline	13
16	Known incidents and decisions not to lose	14
17	Planned dashboard and analytics work	14
18	Planned updater and Windows release architecture	14

18.1	Desired updater flow	15
18.2	Expected release artifacts	15
19	Repository strategy	15
20	Development workflow and collaboration rules	15
21	Current non-negotiable architecture decisions	16
22	Next exact engineering task	16
23	Suggested v4.1 validation matrix	17
24	Continuity handoff for a new conversation	17
A	Current source-map reference	17
B	Feature chronology	18
C	Final decision register	18

1 Executive summary and current state

JAM is a separate product from CV Maker (CVM). Its purpose is to replace scattered spreadsheets, notes and ad-hoc trackers with a polished, local-first Windows desktop application dedicated to the job-search pipeline.

The application currently combines a Python 3.11 backend, a pywebview main shell, local HTML/CSS/vanilla JavaScript, SQLite persistence, a separate Tkinter Capture Mode process, Excel/PDF import-export tooling, portable `.jam` projects, local backups, diagnostics, tracking history, and a local CV/job Analyzer.

Current implementation snapshot

The foundation, Batch 1, Batch 2 and Batch 3 feature sets are implemented. Analyzer v4 is implemented as a hybrid deterministic + local statistical NLP engine, but a real-world French-CV / English-JD test exposed a major multilingual semantic weakness. The next engineering priority is Analyzer v4.1, not packaging or public release.

1.1 Product boundary

JAM and CV Maker must remain separate executables, repositories and data models. JAM may later open or attach CVM files, but it must not become an extension inside CVM.

1.2 Current priorities

1. Preserve the stability of Batch 1-3 while making changes.
2. Fix Analyzer v4 multilingual semantic matching and requirement parsing.
3. Complete useful dashboard analytics after Analyzer accuracy is stable.
4. Only later implement packaging, GitHub repositories, installer and the CVM-style updater workflow.

Important current limitation

Analyzer v4 currently uses TF-IDF word/character similarity and a logistic job-family classifier. This is statistical machine learning, but it is not cross-lingual semantic embedding. In a real test, a French CV and English Business Analyst job description produced incorrect evidence such as English being marked missing despite **English C1** being present in the CV. This defect is documented in detail in Section 11.

2 Product identity and design system

2.1 Brand family

JAM belongs to the same premium product family as CV Maker while remaining visually distinct. The interface direction is permanent dark mode: deep navy/charcoal surfaces, white text, restrained gold accents and limited semantic colors.

2.2 Core palette

Name	Hex	Role
Midnight Navy	#111827	Primary dark surface, headers, identity
Premium Gold	#BC965D	Main accent, active states, premium CTA
Light Gold	#D0AA70	Secondary gold, highlights and borders
White	#FFFFFF	Primary light text / document background
Soft Ice	#F6F8FC	Soft light neutral
Graphite Gray	#374151	Secondary text / dark neutral
Royal Indigo	#4F46E5	Limited interactive/product accent
Emerald Teal	#10B981	Success / positive state
Premium Cream	#F8F6F1	Warm neutral inherited from CVM family
Secondary Cream	#FFFDF9	Warm secondary neutral

2.3 Branding assets

The project uses or expects the following assets under `assets/branding/`:

- `jam_logo.png` - transparent wordmark/monogram usage.
- `jam_icon.png` - square application icon.
- `jam_bg.png` - splash/background asset.
- `jam_runtime.ico` - Windows runtime/taskbar icon used by the main app and Capture Mode.

2.4 UX rules

- Dark theme only unless explicitly changed later.
- Custom modals and polished feedback instead of browser-default alerts.
- Keep tables dense but readable; avoid decorative analytics with no decision value.
- Gold is an accent, not a background for everything.
- Avoid broad permanent MutationObservers that can recursively react to their own DOM changes.
- Small fixes must not silently redesign working screens.

3 Technical architecture

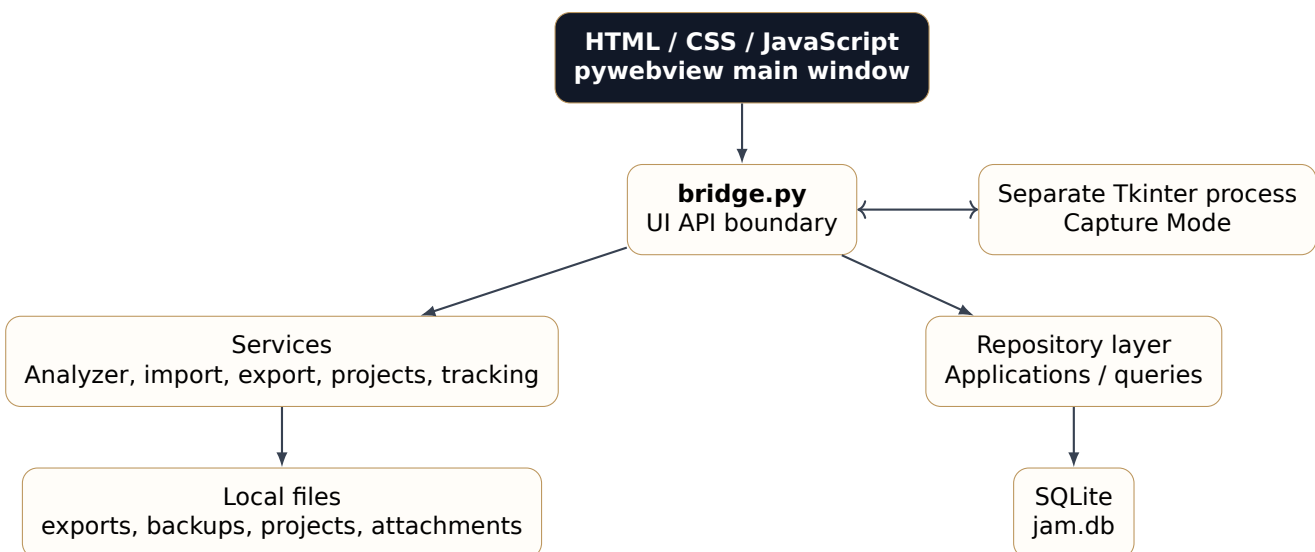
3.1 Technology stack

Desktop shell	Python 3.11 + pywebview
Backend	Python services, bridge, repositories, SQLite
Frontend	Local HTML, CSS and vanilla JavaScript
Database	SQLite with schema migrations
Capture Mode	Separate lightweight Tkinter process
CV parsing	PyMuPDF, python-docx, TXT/MD, readable .cvm
Excel	openpyxl
PDF reports	ReportLab
Images / splash	Pillow
Analyzer v4	Deterministic rule engine + scikit-learn TF-IDF + Logistic Regression
Tests	pytest
Packaging later	PyInstaller + Inno Setup 6
Updater later	GitHub Releases + SHA-256 verification

Current Analyzer v4 requirements include:

```
pywebview>=5.0,<7
PyMuPDF>=1.24,<2
python-docx>=1.1,<2
openpyxl>=3.1,<4
reportlab>=4.2,<5
Pillow>=9,<13
pytest>=8,<9
scikit-learn>=1.5,<2
```

3.2 Runtime architecture



3.3 Separation of responsibilities

Business logic belongs in Python. UI rendering and interaction belong in HTML / CSS / JavaScript. Database rules must not be scattered across frontend files. The bridge is the controlled boundary between the two layers.

4 Runtime data, storage and project portability

4.1 Default runtime locations

The current data-location philosophy is local-first. Defaults are recommended, but the Settings page allows several locations to be changed independently.

Database	%LOCALAPPDATA%/Farouk/JAM/database
Backups	%LOCALAPPDATA%/Farouk/JAM/backups
Settings	%LOCALAPPDATA%/Farouk/JAM/settings
Logs	%LOCALAPPDATA%/Farouk/JAM/logs
Recovery	%LOCALAPPDATA%/Farouk/JAM/recovery
Attachments	%LOCALAPPDATA%/Farouk/JAM/attachments/<application-id>
Exports	%USERPROFILE%/Documents/JAM Exports
Portable projects	%USERPROFILE%/Documents/JAM Projects

The location configuration itself remains under the standard JAM settings area so changing a data directory does not make the configuration impossible to find.

4.2 SQLite schema

The current schema version is **4**. Core tables include:

- applications
- application_status_history
- application_contacts
- application_notes
- application_attachments
- settings
- export_history
- recent_projects
- analyzer_history
- saved_views
- schema_meta

Foreign keys are enabled and child tracking records use `ON DELETE CASCADE`. Database access is wrapped in a context manager with commit/rollback/close semantics.

4.3 Migration philosophy

Migrations are intentionally additive and idempotent. Batch 2 added structured salary columns. Batch 3 preserved old single-contact and notes fields by promoting them into the new contacts and notes history tables only when corresponding richer records do not already exist.

4.4 Portable .jam projects

JAM supports saving/opening a portable .jam project. Save filenames now default to a timestamp so repeated snapshots do not overwrite one another by accident:

```
JAM_DDMYYYY_HHMM.jam  
Example: JAM_23092026_1333.jam
```

The save dialog uses the modern pywebview API **FileDialog.SAVE** rather than the deprecated `SAVE_DIALOG` constant.

Attachment portability rule

Attachment **records** are included in the project state, but the physical attachment files are copied into JAM's local attachment storage and are not embedded byte-for-byte inside every .jam file. This avoids enormous project files. Any future "fully portable archive" feature must be designed explicitly rather than silently changing this behavior.

5 Current application modules

Module	State	Current behavior
Dashboard	Implemented	KPI cards and activity foundation. Rich conversion/source analytics are still planned.
Applications	Implemented	Main table, search, sort/filter, add/edit/delete, multi-select, duplicate handling, quick filters, saved views and bulk actions.
Pipeline	Implemented	Status-oriented board with drag behavior and direct deletion.
Analyzer	Partial	Analyzer v4 exists and exports detailed PDF evidence, but multilingual FR/EN semantics require v4.1.
Exports	Implemented	Excel and PDF exports, branded output, history actions.
Settings	Implemented	Language, salary settings/converter, CV selection, backups, diagnostics, data locations, Capture preferences, reset.
Capture Mode	Implemented	Separate Tkinter process, compact always-on-top capture, remembered position/size, analyzer launch and save.
Guide	Partial	Navigation exists; content remains intentionally incomplete/placeholder.
Updater	Planned	CVM-style updater architecture is explicitly deferred until release work.
Packaging	Planned	PyInstaller + Inno Setup workflow will be built after core product stabilization.

6 Applications and pipeline behavior

6.1 Application fields

The current application record supports, among other fields:

- Company, job title, description, link and location.
- Work mode.
- Status and rating (1-5 stars).
- Source/platform.
- Legacy primary contact name/link plus richer multiple-contact tracking.
- Notes plus notes history.

- Structured salary plus original salary text.
- Saved/applied/updated timestamps.
- Analyzer score and stored analysis JSON.

Current statuses include **Saved, To Review, Applied, Interviewing, Offer, Accepted, Rejected, Withdrawn, Archived**. Archived jobs are hidden by default while remaining searchable when needed.

6.2 Structured salary

Batch 2 introduced structured salary fields:

- minimum / maximum;
- currency;
- period (monthly / annual);
- basis (gross / net);
- original salary text retained for traceability.

The Add/Edit Job currency UI includes local flag assets for common currencies such as EUR, USD and TND.

6.3 Settings salary converter

Settings contains four linked estimate fields:

- Monthly net
- Monthly gross
- Annual net
- Annual gross

Typing in any field recalculates the other three in real time. The current quick estimate uses approximately **net = 78% of gross** and displays a warning that real gross/net conversion varies by country, contract, taxes and deductions. Target salary min/max are identified as **annual gross**.

7 Batch 1 - foundation and safety

Batch 1 established the safety layer that later features must preserve.

- English/French interface switching.
- Local currency flags.
- Database backup creation and restore with a safety backup before restore.
- Add Job autosave/crash recovery.
- Capture Mode autosave/crash recovery.
- .jam autosave/recovery snapshot behavior.
- Dirty-state protection before opening/resetting/closing projects.
- URL normalization.
- Automatic `date_applied` the first time a job enters Applied or a later stage.
- Archive behavior and show/hide archived controls.
- Configurable data-location panel.
- Diagnostics with success/warning/error state presentation.
- Local-first data policy and error/log tooling.

7.1 Notable Batch 1 regressions that were fixed

- Windows backup restore initially failed with `WinError 32` because SQLite file handles could still be open. Connections are now explicitly closed before database replacement.
- The first `i18n` `MutationObserver` reacted to its own text/attribute rewrites and could freeze the interface. The observer was constrained to safer child-list behavior.
- Data-location layout required several iterations to avoid CSS Grid row stretching. The final implementation uses independent left/right settings columns and inserts Data Locations directly in the left stack.

8 Batch 2 - application management and workflow

Batch 2 expanded JAM from a tracker into a more complete application-management tool.

- Duplicate application detection with open-existing / save-anyway / cancel flow.
- Dedicated application details/edit experience.
- Status history timeline.
- Structured salary model.
- Quick filter presets.
- Saved views: save, reopen, rename and delete filter states.
- Bulk actions for selected applications.
- Excel import preview before writing data.
- Excel import error/skip reporting.
- Richer Recent Projects metadata and remove-from-recent behavior.
- Export History actions: open, open folder and remove from history without deleting the actual exported file.
- Logs search and level filters.
- Clickable job/contact URLs.

8.1 Excel import behavior

Excel import is designed to **merge** valid imported rows into the existing JAM Applications table; it does not replace the user's database. Duplicate handling and preview exist to reduce accidental repeated imports.

8.2 Critical Windows import crash and resolution

A runtime crash was observed during Excel import:

```
Tcl_AsyncDelete: async handler deleted by the wrong thread
```

Two contributing areas were addressed:

1. Excel file selection was moved away from a risky pywebview/Tk thread interaction to a native Windows Forms picker running in a separate STA PowerShell process.
2. The Tkinter splash cleanup was hardened so Tk/Tcl objects are released and garbage-collected on the main startup thread before pywebview worker calls can trigger their destruction on a different thread.

This incident is an important regression case for future Windows runtime testing.

9 Batch 3 - richer tracking and Capture improvements

9.1 Multiple contacts

Each application can now hold multiple contacts with:

- name;
- role;
- email;
- phone;
- link;
- note;
- primary/imported marker where applicable.

Contacts can be added, edited, removed and opened. Legacy single contact fields are preserved and promoted into the richer model.

9.2 Notes history

Applications now support timestamped notes history rather than relying only on one static notes field. Notes can be added, edited and removed while preserving creation/update timestamps.

9.3 Attachments

Per-application attachments support categories such as:

- CV
- Cover letter
- Screenshot
- Other

Files are copied into JAM local storage. The UI supports Open, Open Folder and Remove. Attachment deletion only deletes files that are verified to live inside JAM's attachment root.

9.4 Tracking UI placement

The first Batch 3 implementation made the tracking features too hidden behind a separate Tracking view. The UI was then adjusted so the Add/Edit Job workflow clearly exposes a **Tracking & History** area for contacts, notes and attachments, while retaining an optional full tracking view.

The primary Contact Name and Contact Link fields were also cleaned up into two aligned fields on one row, with the Open Contact Link button directly below them at full width.

9.5 Capture Mode persistence

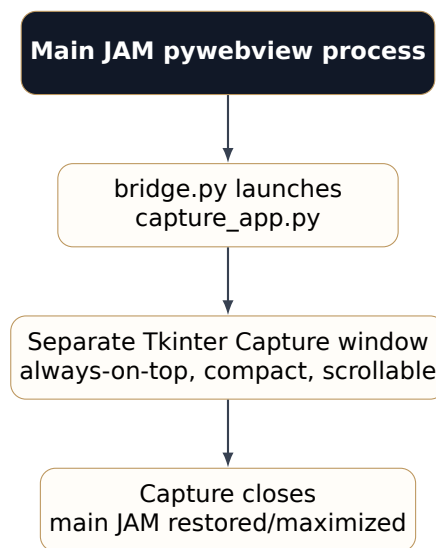
Capture Mode now remembers exact position and size when enabled. Settings allow a default corner (top-left, top-right, bottom-left, bottom-right), a remember-position/size option and a reset control.

Keyboard shortcuts introduced in Batch 3 include:

Ctrl+N	Add Job
Ctrl+K	Focus Applications search
Ctrl+Shift+C	Open Capture Mode
Ctrl+S	Save current .jam project
Ctrl+Enter	Analyze when Analyzer is open
Esc	Close the active dialog

10 Capture Mode architecture

Capture Mode is intentionally a **separate lightweight Tkinter process**. It must not be converted back into a second pywebview window without an explicit architecture decision.



Important Capture-specific decisions:

- The main JAM window minimizes/hides while Capture is open.
- The Capture process uses the same Windows AppUserModelID/icon identity as JAM.
- The PINNED / position selector belongs to scrollable content and should scroll away normally rather than sticking to the viewport.
- Mouse wheel handling must remain smooth and controlled.
- Do not use `unbind_all("<MouseWheel>")` from another window because this previously broke Capture scrolling globally.

11 Analyzer architecture and current defect

11.1 Analyzer v3 foundation

Before v4, JAM used a deterministic, evidence-based rule engine. It parsed and compared:

- role/title family alignment;
- required years of experience;
- salary where reliably detectable;
- required and preferred tools/skills;
- responsibilities/domain concepts;
- languages;

- education level and field;
- explicit CV evidence;
- unrelated-role hard caps.

This rule layer remains valuable because hard facts should not be overridden by fuzzy similarity.

11.2 Analyzer v4 hybrid layer

Analyzer v4 added local statistical NLP while preserving v3 hard checks:

- TF-IDF word n-grams.
- TF-IDF character n-grams.
- Requirement-to-CV-line cosine similarity.
- A local Logistic Regression job-family classifier trained from JAM's curated role taxonomy.
- Requirement coverage percentage.
- Best CV evidence per extracted requirement.
- Evidence labels such as Strong / Partial / Weak / Missing.
- Extended Analyzer PDF reporting.

Nothing is sent to a cloud service.

Why v4 was still useful

The v4 design introduced the correct architectural idea: keep strict deterministic facts, then add a second evidence channel for softer semantic alignment. The failure is not that hybrid scoring is wrong; the failure is that the current semantic channel is primarily lexical and therefore weak across languages.

11.3 Real-world Klépierre test on 23 September 2026

A Business Analyst role at Klépierre was tested against the user's French CV. The job description strongly matched performance reporting, dashboards, Excel, Power BI, CRM/data tools, data quality, AI interest, a Master's degree and fluent English.

Observed Analyzer v4 output included:

- overall score: approximately **65%**;
- requirement coverage: only **9%**;
- predicted family: Performance Analyst;
- engine shown as **lexical fallback**;
- "Fluent English is essential" incorrectly marked Missing at roughly 10%;
- target role marked Weak around 27% even though the CV headline contains Business Data Analyst / Analyste Performance & Data;
- education interpreted too narrowly as Engineering even though the JD accepts Business School, Engineering School or University;
- "Prepare management reports, dashboards, and analytical reviews" marked missing/very weak despite explicit Power BI/Excel dashboard work in the CV.

The CV itself explicitly contains **English C1**, a Master 1 in Business Analytics, a Master 2 data/analytics program, performance/KPI/reporting experience, Power BI, Excel, Salesforce and data-quality evidence. Therefore the low semantic coverage is not a realistic reading of the match.

11.4 Root cause

The main failure is a language mismatch:

Current v4 behavior

French CV phrase: “analyse de performance / reporting décisionnel / fiabilisation des données”
English JD phrase: “performance analysis / management reporting / data quality enhancement”

TF-IDF can only exploit overlapping tokens/character patterns. It does not truly know that those French and English phrases mean the same thing. In addition, raw CV section headings can currently be selected as evidence, and requirement extraction can over-consume company marketing text or headings.

11.5 Analyzer v4.1 - next required fix

The next Analyzer iteration should be a controlled rebuild of the semantic/evidence layer, not a collection of one-off phrase patches.

1. **Real multilingual sentence embeddings.** Use an offline multilingual sentence-embedding model so French and English meaning can be compared directly. The exact model must be selected based on accuracy, package size and Windows bundling constraints; do not hard-code a model choice before benchmarking it.
2. **Section-aware CV parser.** Recognize sections such as Profile, Experience, Education, Skills, Projects and Languages. Language requirements must search language evidence first. Education requirements must search education first.
3. **Evidence filtering.** Section headings like “EXPÉRIENCES PROFESSIONNELLES” must never be returned as the best semantic evidence line.
4. **Requirement typing.** Distinguish mandatory, preferred/appreciated, responsibility, domain context and soft-skill statements.
5. **Alternative education parsing.** Understand “Business School OR Engineering School OR University” as alternatives instead of collapsing it to one field.
6. **Hard-fact precedence.** If the CV explicitly states English C1, a fuzzy semantic layer must not mark English as missing.
7. **Scoring rebuild.** Semantic evidence should support factual scoring rather than overriding explicit structured evidence.
8. **Regression corpus.** Add fixed tests using representative Data/BI, Legal, Accounting, Data Engineering and Business Analyst examples, including French-CV / English-JD pairs.

A reasonable target score architecture to evaluate is:

Role / domain alignment	20%
Hard skills / tools	25%
Responsibilities	20%
Experience	10%
Education	10%
Languages	5%
Semantic evidence support	10%

These weights are a design target, not a frozen contract. Calibration should be driven by regression cases, not intuition alone.

11.6 Dataset strategy

A giant CSV containing hundreds of thousands or millions of job offers should **not** simply be bundled into JAM. Raw volume does not automatically improve scoring and would bloat the installer.

A better future strategy is:

- use occupational taxonomies such as ESCO/O*NET or a curated public job corpus for training/evaluation;
- train or tune classifiers offline during development;
- ship only the compact artifacts needed at runtime;
- preserve local/private user analysis;
- benchmark any added dataset against a fixed regression suite before keeping it.

12 Analyzer report output

The Analyzer PDF is intended to be independently reviewable. It includes:

- JAM branding and title;
- user-entered job title;
- full job description;
- detected job reference when available;
- overall score and label;
- score breakdown;
- experience, education and salary extraction;
- matched/missing requirements;
- notes/review guidance;
- CV evidence;
- local ML evidence in v4;
- a disclaimer that the Analyzer can make mistakes and external verification is encouraged.

A previous ReportLab failure occurred when a very long job description was placed inside a single non-splittable table cell. The report service was adjusted so long descriptions can flow across pages normally.

13 Import, export and project workflow

13.1 Excel export

Excel output uses premium styling, centered titles, real Excel datetime values and normal autofiltering. A previous combination of Structured Table + AutoFilter caused Excel repair warnings, so the Structured Table dependency was removed from the export design.

13.2 Excel import

Import preview should show detected rows and potential issues before database changes. Valid rows can be imported while bad/duplicate rows are reported separately.

13.3 PDF export

JAM PDF exports use large, correctly-proportioned branding rather than a tiny/crushed logo. Analyzer reports use the same premium identity.

13.4 Export history

Export history supports Open, Open Folder and Remove from History. Removing a history entry does not silently delete the actual file from disk.

14 Settings, internationalization and diagnostics

14.1 Language

English and French are available from Settings. The language card is intentionally compact and includes local flag assets. New dynamic content must be translated, not only static HTML.

14.2 Data Safety

Settings provides backup creation/restore and backup-folder access. Restore must not replace an open/locked SQLite database file. The system creates a safety backup before restoring another backup.

14.3 Data Locations

The Data Locations panel lets the user inspect and edit several storage destinations. UI rule: each path and its Open/Edit actions stay on one line where space permits, and the panel must size to its content rather than inheriting the height of a taller neighboring settings card.

14.4 Diagnostics

Diagnostics use clear success/warning/error states. The diagnostic result list scrolls independently while the action buttons remain visible.

15 Testing and regression discipline

Automated tests are important, but runtime Windows behavior remains authoritative.

Current test-state note

Batch 3 was reported at 59 passing tests. Analyzer v4 added three dedicated hybrid tests, making the expected merged count 62. This exact merged count should be re-run on the developer machine after any v4.1 work; do not rely on an old assistant-side count as proof of the current local checkout.

Every future bug fix should add or extend a regression test when practical. Critical manual checks include:

- launch/splash/main-window lifecycle;
- Capture open/close, position, size, icon and scrolling;
- Excel import without app termination;
- save/open .jam;
- dirty-state prompts and recovery;
- application add/edit/delete and status timeline;
- contacts/notes/attachments persistence;
- EN/FR switching;

- Analyzer score/evidence against known regression examples;
- PDF/Excel exports opening cleanly in real applications.

16 Known incidents and decisions not to lose

Incident / decision	Cause / risk	Current rule / resolution
Capture as second pywebview window	Heavy / fragile behavior	Keep separate Tkinter process architecture.
Mouse wheel dies after Analyzer closes	Global Tk <code>unbind_all</code> removed bindings	Never use global unbind that destroys Capture wheel handling.
Python taskbar icon in Capture	Separate process lacked Windows identity/icon handling	Set JAM AppUserModelID and native big/small icon.
Excel import terminates JAM	Tcl/Tk objects destroyed on wrong thread	Native Windows file picker + explicit main-thread splash cleanup.
Deprecated pywebview dialog constants	Future compatibility warning	Use <code>FileDialog.SAVE</code> ; modernize other dialogs where safe.
Save project overwrites generic name	Constant <code>JAM_Project</code> default	Timestamp default <code>JAM_DDMMYYYY_HHMM.jam</code> .
Excel export repair warning	Structured Table + AutoFilter conflict	Keep plain autofilter with styling.
Settings Data Locations gap/stretch	CSS Grid row height coupled columns	Use independent left/right stacks; insert Data Locations in left column.
French/English Analyzer mismatch	TF-IDF lexical similarity is not multilingual semantics	Analyzer v4.1 multilingual embedding + parser rebuild.

17 Planned dashboard and analytics work

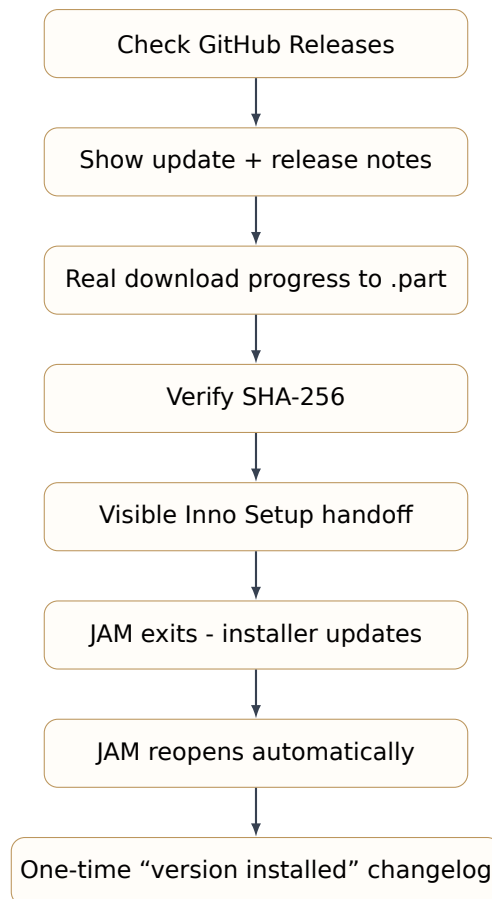
After Analyzer accuracy is stable, the remaining intelligence/analytics work includes:

- better dashboard KPIs;
- two small useful charts rather than decorative chart overload;
- applications over time;
- status conversion metrics;
- source/platform analytics;
- average Analyzer score by source where useful;
- interview / offer conversion by source;
- practical summaries that support job-search decisions.

18 Planned updater and Windows release architecture

This work is explicitly **deferred until the user says to start release/update work**. It should mirror the proven CVM experience.

18.1 Desired updater flow



The UI should include both automatic detection and a manual **Check for Updates** action. When an update is available, the user should see what is fixed/added before choosing to install. During update, the app must explicitly say not to close/reopen it manually. After restart, a one-time success window should show the installed version and a checklist of changes.

18.2 Expected release artifacts

```
JAM-Setup-X.Y.Z.exe  
JAM-vX.Y.Z-Windows-x64.zip  
SHA256.txt
```

19 Repository strategy

JAM should follow the same separation discipline proven in CVM:

- **Source repository:** complete development project and code.
- **Public distribution repository:** README, releases, installer/ZIP/checksum only.

Likely names are `JAM-Source-Code` and `JAM`, but repository creation is deferred and names should be confirmed before pushing anything.

Never push the full source tree into the public user-download repository by accident.

20 Development workflow and collaboration rules

The preferred working style for future conversations is strict and incremental.

- Diagnose the real problem before editing.
- Preserve working architecture; do not rewrite unrelated systems during a targeted fix.
- Prefer complete-file replacements over manual snippet merging.
- Always state the exact target path for every replacement file.
- **JavaScript rule:** provide full JavaScript as .txt artifacts for copy/paste into the target .js; do not deliver downloadable .js artifacts.
- After each update, provide a short checklist of exactly what to test.
- Run/validate syntax/tests before handing over changes.
- Keep normal replies short unless the user explicitly requests documentation or full code.
- Do not move into packaging/release work until the source state is stable and the user explicitly asks.

Developer commands:

```
cd C:\Users\fdame\Desktop\JAM
.\.venv\Scripts\Activate.ps1
python src\main.py
pytest -q
```

21 Current non-negotiable architecture decisions

1. JAM stays separate from CVM.
2. Windows desktop, local-first and privacy-focused.
3. Python backend + pywebview frontend shell + SQLite.
4. Capture Mode remains a separate Tkinter process.
5. Database and attachment data stay local by default.
6. .jam remains the portable project format.
7. EN/FR must remain synchronized across new features.
8. Analyzer scoring must be explainable with visible evidence.
9. Unrelated professions must remain hard-capped even after semantic upgrades.
10. Public release/updater system comes later and must use GitHub Releases, checksums and separate repos.

22 Next exact engineering task

Priority for the next conversation

Analyzer v4.1: replace the lexical-only cross-language weakness with a real multilingual semantic layer, add section-aware CV/JD parsing, prevent headings from becoming evidence, fix education alternatives, give explicit hard facts precedence (e.g. English C1), recalibrate scoring, and add regression cases based on the Klépierre Business Analyst test plus unrelated Legal/Accounting controls.

Do not begin by redesigning the UI, packaging the application, creating GitHub repositories or adding a giant job-offer CSV. First make the Analyzer reliable on realistic multilingual cases without regressing the deterministic protections already working.

23 Suggested v4.1 validation matrix

Case	Expected behavior	Reason
Klépierre Group Leasing Business Analyst vs current French CV	Strong/moderate-high fit; English, Master's, BI/reporting/data quality recognized; leasing/real-estate gaps remain visible	Valid cross-language Business Analyst regression
Juriste droit social requiring Master 2 law + 7 years	Very low score / hard cap	Profession and education field mismatch
Accounting role requiring accounting degree/standards	Low score	Generic Excel/reporting must not override missing accounting fundamentals
Data Analyst requiring SQL/Power BI/Excel/data quality	High score when evidence exists	Core target family
Data Engineer requiring Snowflake pipelines/cloud stack	Moderate/low if missing core platform evidence	Related data family but material skill gaps
English required with CV Languages: English C1	Explicit match, never semantic Missing	Structured hard fact precedence

24 Continuity handoff for a new conversation

When this PDF is uploaded into a new conversation, the assistant should treat it as the current project handoff and follow this order:

1. Confirm that the existing Batch 1-3 architecture must be preserved.
2. Treat Analyzer v4 multilingual accuracy as the immediate open problem.
3. Ask for the latest relevant source files only if the actual file bytes are needed to implement a fix; do not reconstruct them from memory.
4. Make the smallest synchronized set of changes.
5. Return complete replacement files, with JavaScript as .txt.
6. Provide exact Windows target paths and a short runtime test checklist.
7. Re-run the full regression suite and manually validate the Klépierre case.

Short continuity statement

JAM is already a substantial working application. The job is no longer to invent its architecture from scratch. Preserve what works, fix Analyzer v4.1 carefully, then complete Batch 4 analytics and only afterward move to packaging/release/updater work when the user explicitly asks.

A Current source-map reference

The following list is intentionally not a promise that every file is unchanged in the developer checkout; it is the known architecture reference from the latest delivered stages.

Path	Role
src/main.py	Main application startup
src/config.py	Central application/runtime configuration
src/startup.py	Splash/startup identity and Tk cleanup
src/bridge.py	pywebview API boundary and Windows dialogs/actions
src/capture_app.py	Separate Tkinter Capture Mode application
src/database.py	SQLite schema, migrations, connection lifecycle
src/repositories/application_repository.py	Application persistence/query layer
src/services/import_service.py	Excel import/preview logic
src/services/export_service.py	Excel/PDF export logic and history
src/services/project_service.py	Portable .jam save/open
src/services/settings_service.py	Settings persistence
src/services/saved_view_service.py	Saved filter/view persistence
src/services/application_tracking_service.py	Contacts, notes history and attachment storage
src/services/analyzer_service.py	Deterministic Analyzer v4 core
src/services/semantic_match_service.py	TF-IDF + job-family classifier semantic layer
src/services/analyzer_report_service.py	Analyzer PDF report generation
ui/index.html	Main application markup
ui/css/app.css	Main product styling
ui/js/app.js	Core UI behavior
ui/js/il8n.js	EN/FR translation layer
ui/js/batch1.js	Foundation/safety UI additions
ui/js/batch2.js	Application-management UI additions
ui/js/batch3.js	Tracking/Capture/shortcut UI additions
ui/js/analyzer_hotfix.js	Analyzer UI/reporting integration layer
assets/branding/*	JAM logo/icon/splash identity
tests/*	Regression and feature tests

B Feature chronology

Foundation	Desktop shell, premium UI, applications, pipeline, Capture, Analyzer baseline, exports, .jam, backups/logs/support.
Batch 1	Safety, recovery, EN/FR, data locations, diagnostics, archive/date logic, local flags.
Batch 2	Duplicate detection, detail/timeline, structured salary, saved views, bulk actions, Excel preview/errors, richer history/actions.
Batch 3	Multiple contacts, notes history, attachments, Capture exact geometry, shortcuts, Tracking UI.
Analyzer v4	TF-IDF/character similarity, local Logistic Regression role classifier, requirement evidence and report expansion.
Next: v4.1	True multilingual semantic layer + section-aware parsing + scoring/evidence repair.
Later Batch 4	Dashboard/source/conversion analytics after Analyzer stabilization.
Release phase	PyInstaller, Inno Setup, separate GitHub repositories, CVM-style updater.

C Final decision register

- Current development version remains 0.1.0 until deliberately changed.

- The product has not reached a public JAM release yet.
- Do not imply the updater is already implemented; it is a later planned milestone.
- Do not bundle a million-row job CSV merely because “ML” is desired.
- The Analyzer must stay explainable and evidence-backed.
- Multilingual semantic accuracy is now a blocker for trusting v4 scores.
- Keep the user’s existing data safe during every migration.
- Preserve the premium JAM/CVM family identity and permanent dark app UI.

JAM - Job Application Manager

Technical continuity snapshot - 23 September 2026